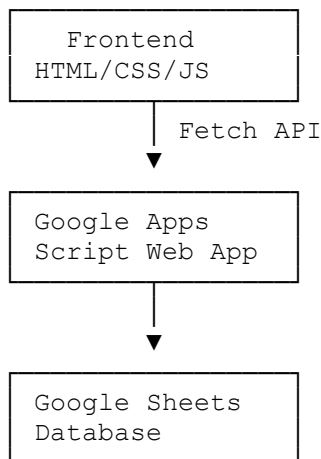


How SNACKS SPOT Was Built

SNACKS SPOT is a lightweight Point of Sale (POS) and billing system designed for small restaurants, tea shops, and snack bars. The project was built using a serverless architecture where Google Sheets acts as the database and Google Apps Script acts as the backend API.

This approach eliminates the need for traditional databases, hosting servers, or backend frameworks.

Architecture Overview



The frontend communicates with Google Apps Script through HTTP requests.

Google Apps Script performs all data operations and stores information in Google Sheets.

Step 1: Design the Data Structure

Before writing any code, the sheets were designed to represent the application's data.

Menu Sheet

Stores menu items.

ID	ItemName	Price	Image	Status
1	Idly	10	URL	Active

Purpose:

- Display menu items
 - Manage prices
 - Enable or disable items
-

Orders Sheet

Stores completed orders.

OrderID	Date	Items	Total
ORD123	Timestamp	JSON	120

Purpose:

- Order history
 - Sales reporting
 - Revenue calculations
-

Settings Sheet

Stores application settings.

Key	Value
UPI_ID	snacksspot@upi

Purpose:

- Centralized configuration
 - Change settings without modifying code
-

Admin Sheet

Stores login credentials.

Username	Password
admin	password

Purpose:

- Basic admin authentication

Step 2: Create the Google Apps Script Project

A new Apps Script project was created and connected to the spreadsheet.

```
const SPREADSHEET_ID = "YOUR_SPREADSHEET_ID";
```

Helper function:

```
function getSpreadsheet() {  
    return SpreadsheetApp.openById(SPREADSHEET_ID);  
}
```

Purpose:

- Avoid repeating spreadsheet access code

Step 3: Build API Responses

All responses are returned as JSON.

```
function jsonResponse(data) {  
    return ContentService  
        .createTextOutput(JSON.stringify(data))  
        .setMimeType(ContentService.MimeType.JSON);  
}
```

Purpose:

- Standard API format
- Easy consumption from JavaScript

Step 4: Create GET Endpoints

The application exposes read-only APIs using `doGet()`.

```
function doGet(e)
```

Supported actions:

Action	Purpose
getMenu	Load menu
getSettings	Load settings
getOrders	View orders
getReport	Sales report

Example:

```
?action=getMenu
```

Returns:

```
{
  "success": true,
  "menu": [...]
}
```

Step 5: Create POST Endpoints

Write operations are handled using `doPost()`.

```
function doPost(e)
```

Supported actions:

Action	Purpose
login	Authenticate admin
addMenu	Create item
updateMenu	Edit item
deleteMenu	Remove item
saveOrder	Save customer order

This creates a simple REST-style API.

Step 6: Build Menu Management

Menu items are stored in Google Sheets.

Loading menu:

```
getMenu()
```

Returns:

```
{
  id,
  name,
  price,
  image,
  status
}
```

Frontend requests menu data and dynamically renders cards.

Example card:

Idly
₹10

[Add]

Benefits:

- No hardcoded menu
- Menu changes instantly reflect in UI

Step 7: Build Cart Functionality

The frontend maintains a local cart.

```
let cart = [];
```

Adding an item:

```
addToCart(name, price)
```

Logic:

1. Check if item already exists.
2. Increase quantity if present.
3. Add new item if absent.
4. Recalculate bill.

Example:

```
{
  name: "Idly",
  price: 10,
  qty: 2
}
```

Step 8: Generate Bill

Whenever the cart changes:

```
renderCart()
```

runs.

It calculates:

```
price × quantity
```

for every item.

Example:

```
Idly × 2 = ₹20
```

```
Tea × 1 = ₹15
```

```
Total = ₹35
```

Step 9: Add UPI QR Payments

The system fetches the UPI ID from the Settings sheet.

```
settings.UPI_ID
```

Generates:

```
upi://pay
```

payment URL.

Example:

```
upi://pay?pa=shop@upi&pn=SNACKS SPOT&am=120
```

The URL is converted into a QR code image.

Customer scans and pays using any UPI app.

Step 10: Save Orders

When the cashier clicks:

```
Place Order
```

the frontend sends:

```
{  
  action: "saveOrder",  
  items: cart,  
  total: total  
}
```

Apps Script:

```
saveOrder()
```

creates:

```
ORD1748537261
```

and stores:

```
Order ID  
Date  
Items  
Total
```

in the Orders sheet.

Step 11: Print Bills

The system generates a small printable HTML page.

```
window.open()
```

creates a popup.

```
window.print()
```

sends it to the printer.

This avoids requiring external libraries.

Step 12: Build Reports

Reports are generated directly from the Orders sheet.

```
getSalesReport()
```

Calculates:

Total Orders
Total Revenue

Example:

```
{  
  "totalOrders": 42,  
  "totalRevenue": 8760  
}
```

Step 13: Deploy as a Web App

Apps Script deployment:

1. Deploy
2. New Deployment
3. Web App
4. Anyone access
5. Deploy

Generated URL:

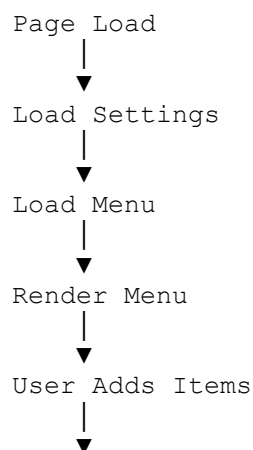
`https://script.google.com/macros/s/...`

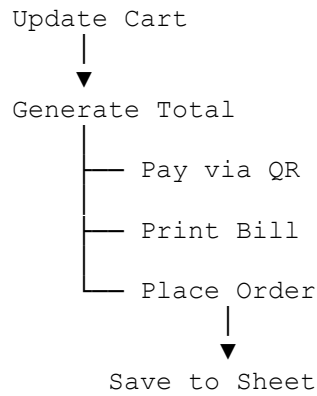
Frontend uses:

```
const API_URL = "WEB_APP_URL";
```

All communication happens through this endpoint.

Frontend Workflow





Why This Approach Was Chosen

Low Cost

- No database hosting
- No backend server
- No cloud infrastructure

Easy Maintenance

- Data can be edited directly in Google Sheets
- Non-technical users can update menu items

Fast Development

- Apps Script provides backend functionality immediately
- No need for Node.js, PHP, or Java

Suitable For

- Tea shops
 - Snack bars
 - Small restaurants
 - Cafeterias
 - College canteens
 - Temporary food stalls
-

Future Enhancements

Potential improvements include:

- Menu categories
- Quantity increase/decrease controls
- Customer information
- Kitchen Order Tickets (KOT)
- GST calculations
- Dashboard analytics
- Inventory management
- Table management
- Multi-user authentication
- Progressive Web App support
- Offline functionality

This project demonstrates how a complete POS system can be built using only HTML, CSS, JavaScript, Google Apps Script, and Google Sheets, without requiring a traditional backend server or database.